# Lesson: C Fundamentals

CSE 4107 : Structured Programming I

Shahriar Ivan

# Writing a Simple Program

Your first C program: “Hello, world”

```
#include <stdio.h>
int main(void){
    printf(“Hello, world\n”);
    return 0;
}
```

- The program can be saved in a file named *hello.c*
- You can name it anything, but the *.c* extension is often required

# Compiling and Linking

- Before a program can be executed, three steps are usually required
    - ***Preprocessing.*** The ***preprocessor*** obeys commands that begin with # (known as ***directives***)
    - ***Compiling.*** A ***compiler*** translates the program into machine code (***object code***)
    - ***Linking.*** A ***linker*** combines the object code produced by the compiler with any additional code needed (like, ***printf***) to yield a complete executable program
- The preprocessor is usually integrated with the compiler

In the command line: cc -o hello hello.c or gcc -o hello hello.c

# The GCC Compiler

- One of the most popular C compiler
- Supplied with Linux, but available for many other platforms

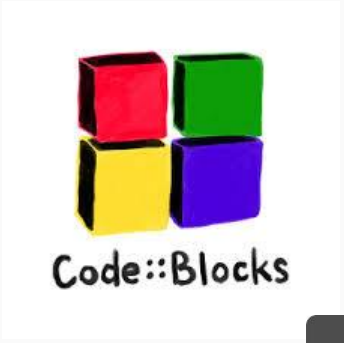
In the command line: gcc -o hello hello.c

# Integrated Development Environments

- Alternative to ***command line*** compiler
- Supplied with Linux, but available for many other platforms
- A software package that allows:
    - Edit
    - Compile
    - Link
    - Execute
    - Debug
    - Syntax Highlighting

# Integrated Development Environments

Popular IDEs are:



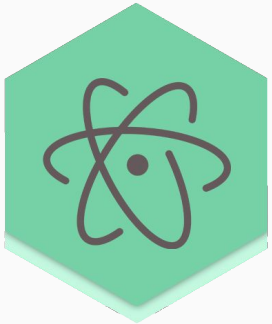










# General form of a Simple Program

Your first C program: “Hello, world”

```
#include <stdio.h>
int main(void){
    printf(“Hello, world\n”);
    return 0;
}
```

```
directives
int main(void){
    statements
}
```

# General form of a Simple Program

- C uses ***{*** and ***}*** like other languages uses ***begin*** and ***end***
- Three key features:
  - Directives (***editing commands*** that modify program prior to compilation)
  - Functions (named ***blocks*** of executable codes)
  - Statements (***commands*** to be performed when the program is run)

# General form of a Simple Program

*Directives*

- A preprocessor edits a C program before it is compiled
- Commands intended for the preprocessor are called directives
  - Example: `#include <stdio.h>`
- `<stdio.h>` is a ***header*** containing information about C's standard ***I/O*** library
- Begins with a **#** character
- One line long
- No semicolon or other special characters at the end

# General form of a Simple Program

*Functions*

- Named block of executable code
- Building blocks from which programs are constructed
- Concept came from mathematics
  - f(x) -> x+1
- Functions which computes values uses *return* statement to specify what value it **returns**
- Two categories:
  - User defined functions
  - Provided as a part of C implementation -> Library functions

# General form of a Simple Program

*Functions*

The *main* function:

- The *main* function is mandatory
- Gets automatically called when the program is executed
- Returns a status code; the value 0 means normal program execution
- Many compilers will produce warning messages if no *return* statement is provided at the end of *main* function

# General form of a Simple Program

## *Statements*

- A ***statement*** is a command to be executed when the program runs
  - Example from *hello.c*:
  - printf(“Hello, world\n”);
- Our previous program ***hello.c*** uses only two types of statements:
  - ***return*** statement
  - function call -> Asking a function to perform its assigned task
- C requires that each statement end with a semicolon
  - There’s one exception: the compound statement

# General form of a Simple Program

## *Printing Strings*

- *printf* displays ***string literal*** - characters enclosed in double quotes
- *printf* doesn't automatically advance to the next output line
- To make *printf* advance one line, use *\n*
- The statement -> printf("Hello, world\n");
  - Can be replaced by two calls of *printf* :
  - printf("Hello,");
  - printf("world\n");

# Comments

*C89 Style*

- A ***comment*** begins with */** and ends with ***/**

```
/*
Name: hello.c
Purpose: Prints Hello, world
Author: Mohammed Saidul Islam
*/
#include <stdio.h>
int main(void){    /* beginning */
        printf("Hello, world\n");    /* print function */
        return 0;
}
```

- Can appear in anywhere in the program
- Comments may extend over more than one line

# Comments

*C89 Style*

```
/*
Name: hello.c
Purpose: Prints Hello, world
Author: Mohammed Saidul Islam

#include <stdio.h>
int main(void){    /* beginning */
        printf("Hello, world\n");    /* print function */
        return 0;
}
```

- Failure to end a comment can result in program statements being commented out, causing your program to be incorrect.

# Comments

*C99 Style*

- ***comment*** begins with ***//***

```
// Name: hello.c
// Purpose: Prints Hello, world
// Author: Mohammed Saidul Islam

#include <stdio.h>
int main(void){    // beginning
        printf("Hello, world\n");    // print function
        return 0;
}
```

- Less error prone : no chance of unterminated comment to consume part of a program
- Simple and helpful because it ends automatically at the end of the line
- Multiline comments stands out better

# Variables and Assignment

*Types*

- Most programs need a way to store data temporarily
- These storage locations are called ***variables***
- Each variable has a ***type*** -> specifies the type of data it will store
- Example:
  - int
    - Short for *integer*, E.g.: 0, -243, 243
    - Can store whole numbers
    - Limited range

# Variables and Assignment

*Types*

- Example:
  - float
    - Short for *floating point* numbers
    - Can store much larger numbers than *int*
    - Can stores numbers with digits after the decimal point, like 379.125
    - Limitations are:
      - Slower arithmetic operations
      - Approximate nature of *floating point* values (i.e., round-off error)

# Variables and Assignment

*Declarations*

- Variables must be declared before they are used
- Variables can be declared one at a time:
  - `int height;`
  - `float profit;`
- Alternatively, several variables can be declared at the same time:
  - `int height, length, width, volume;`
  - `float profit, loss;`

# Variables and Assignment

*Assignment*

- ***Assignment*** is the process of assigning a value to a variable.
  - height = 8;
  - profit = 100.0;
- Declare first, then assign

```
// Correct way
int height;
height = 8;
int length = 12, width = 10;
float profit = 100.0;
```

```
// Wrong way
height = 8;
int height;
// Not wrong, but be careful
int length, width = 10;
```

# Variables and Assignment

## *Assignment*

- It's best to append the letter f to a floating point constant
  - `float profit = 100.0f;`
- Assigning *int* to *float*, or vice versa is possible but not recommended
- Once assigned, a variable can be used to compute the value of another variable:
  - `int height = 10, length = 20, width = 5, volume;`
  - `volume = height * length * width; // volume is now 1000`
- The right side of an assignment can be a formula ( or expression in C terminology) involving constants, variables, and operators

# Variables and Assignment

*Printing the value of a variable*

- *printf* can be used to display the current value of a variable
- To write the message
  - Height: *h*

    where *h* is the current value of the height variable, we'd us the following call of *printf*

    printf("Height: %d\n", height);
  - *%d* is a placeholder indicating where the value of *height* is to be filled in
  - *%d* works for *int*, *%f* is required for *float*

# Variables and Assignment

*Printing the value of a variable*

- By default, *%f* displays a number with six digits after the decimal points
- To force *%f* to display *p* digits after the decimal point, put *.p* between % and f
- To print the line
  - Profit: $2150.48

    We have to use the following printf:

    printf("Profit: $%.2f\n", profit);

# Variables and Assignment

*Printing the value of a variable*

- There’s no limit to the number of variables that can be printed by a single call of *printf*:
  - printf(“Height: %d Length: %d\n”, height, length);

# Program

*Computing the dimensional weight of a box*

- Shipping companies often charge extra for boxes that are large, but very light based on their volume instead of weight
- Usual method followed in the US:
  - Divide the volume of the box by 166 (the allowed number of cubic inches per pound)

    -> Dimensional weight
  - If the value exceeds its actual weight, shipping fee is based on dimensional weight
- We'll write a program, that computes the dimensional weight of a particular box:
  - Dimension: 12" x 10" x 8", Volume: 960 (cubic inches), Dimensional weight: 6

# Program

*Computing the dimensional weight of a box*

- Division is represented by / in C:
  - `weight = volume  / 166`
- In C, when one integer is divided by another, the answer is truncated
  - The volume will be 960 cubic inches
  - Dividing by 166 gives 5, instead of 5.783
- We'll write a program, that computes the dimensional weight of a particular box:
  - Dimension: 12" x 10" x 8", Volume: 960 (cubic inches), Dimensional weight: 6

# Program - Initialization

*Computing the dimensional weight of a box*

- Some variables are automatically set to zero when a program begins to execute, but most are not
- A variable which doesn't have a default value and hasn't yet been assigned a value, is said to be ***uninitialized***

**Note:** Attempting to access the value of an uninitialized variable may yield an unpredictable result. With some compilers, worst-case behavior - even a program crash - may occur.

# Reading Input

- scanf is the C library's counterpart to printf
- scanf requires a ***format string*** to specify the appearance of the input data
- Example of using scanf to read an int value:
  - scanf("%d", &i) //reads an integer and stores it into *i*
- To read a float value:
  - scanf("%f", &x) //reads a float and stores it into *x*
- The & symbol is usually (but not always) required when using scanf

# Program (Revisited)

*Computing the dimensional weight of a box*

- We can modify our previous program to take inputs from user using scanf
- Each call of scanf is immediately preceded by a call of printf that displays a ***prompt.***
- Sample output of the program:

```
Enter height of the box: 8
Enter length of the box: 12
Enter width of the box: 10
Volume (cubic inches): 960
Dimensional weight (pounds): 6
```

# Defining Names for constants

- Both of the programs rely on the constant 166, whose meaning may not be clear to someone reading the program.
- Using a feature known as ***macro definition,*** we can name this constant:

  ```
  #define INCHES_PER_POUND 166
  ```
- When a program is compiled, the preprocessor replaces each macro by the value that it represents

# Defining Names for constants

- During preprocessing, the statement

  ```
  weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
  ```

  will become

  ```
  weight = (volume + 166 - 1) / 166;
  ```

- The value of a macro can be an expression:

  ```
  #define RECIPROCAL_OF_PI (1.0f / 3.14159f)
  ```

- If it contains operators, the expression should be enclosed in parentheses and using only uppercase letters in macro names is a common convention

# Program

*Converting from Fahrenheit to Celsius*

- Given a temperature in Fahrenheit, convert it to Celsius
  - Formula: C = (F - Freezing Point) x Scale Factor
    = (F - 32) x (5/9)
- Sample program output:

  Enter Fahrenheit temperature: 212

  Celsius equivalent: 100.0
- This program will allow temperatures which are not integers

# Program - Analysis

*Converting from Fahrenheit to Celsius*

- Defining `SCALE_FACTOR` to be `(5.0f/9.0f)` instead of `(5/9)` is important
- Note the use of `%.1f` to display celsius with just one digit after decimal

# Identifiers

- Names for variables, functions, macros, and other entities are called ***identifiers.***
- An identifier may contain letters, digits, and underscores, but must begin with a letter or underscore:

  `times10` `get_next_char` `_done`

  It's usually best to avoid identifiers that begin with an underscore.
- Examples of illegal identifiers:

  `10times` `get-next-char`

# Identifiers

- C is ***case-sensitive:*** it distinguishes between upper-case and lower-case letters in identifiers
- For example, the following identifiers are all different:
  - job joB jOb jOB Job JoB JOb JOB
- Many programmers use only lower-case letters in identifiers (other than macros), with underscores inserted for legibility:

  symbol_table current_page name_and_address

# Identifiers

- Other programmers use an upper-case letter to begin each word within an identifier:

  symbolTable  currentPage  nameAndAddress

- C places no limit on the maximum length of an identifier

# Identifiers

*Keywords*

- The following ***keywords*** can't be used as identifiers:

| | | | |
|---|---|---|---|
| auto | enum | restrict* | unsigned |
| break | extern | return | void |
| case | float | short | volatile |
| char | for | signed | while |
| const | goto | sizeof | _Bool* |
| continue | if | static | _Complex* |
| default | inline* | struct | _Imaginary* |
| do | int | switch | |
| double | long | typedef | |
| else | register | union | |